FREE REFERENCE SHEET

Azure Zero-Trust Terraform Cheat Sheet

8 production-ready patterns for NSGs, Private Endpoints, Managed Identities & Forced Tunneling

woitzik.dev
cloud infrastructure engineering

1: NSG DEFAULT-DENY BASELINE

Deny-All Outbound (ISO 27001 baseline)

```hcl
resource "azurerm_network_security_rule" "deny_all_out" {
  name                        = "DenyAllOutbound"
  priority                    = 4096
  direction                   = "Outbound"
  access                      = "Deny"
  protocol                    = "*"
  source_port_range           = "*"
  destination_port_range      = "*"
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  resource_group_name         = var.resource_group_name
  network_security_group_name = azurerm_network_security_group.spoke.name
}
```

Deny-All Inbound + Allow VNet Internal

```hcl
# 4095: allow internal VNet traffic
resource "azurerm_network_security_rule" "allow_vnet_in" {
  priority                   = 4095
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "*"
  source_address_prefix      = "VirtualNetwork"
  destination_address_prefix = "VirtualNetwork"
  # ... (name, rg, nsg_name)
}
# 4096: deny everything else
resource "azurerm_network_security_rule" "deny_all_in" {
  priority  = 4096
  direction = "Inbound"
  access    = "Deny"
  # source/dest = "*"
}
```

Rule: Always set 4096 deny-all as the last rule. Never rely on Azure's implicit deny. Make it explicit for audit evidence.

2: PRIVATE ENDPOINT + AUTOMATIC DNS

Key Vault: full Private Link stack (works identically for Storage, ACR, SQL)

```
resource "azurerm_private_endpoint" "kv" {
  name                = "pe-kv-${var.environment}"
  location            = var.location
  resource_group_name = var.resource_group_name
  subnet_id           = azurerm_subnet.private_endpoints.id

  private_service_connection {
    name                           = "psc-kv"
    private_connection_resource_id = azurerm_key_vault.main.id
    subresource_names              = ["vault"]     # "blob", "sqlServer", "registry"
    is_manual_connection           = false
  }

  private_dns_zone_group {
    name                 = "kv-dns-group"
    private_dns_zone_ids = [azurerm_private_dns_zone.kv.id]
  }
}

resource "azurerm_private_dns_zone" "kv" {
  name                = "privatelink.vaultcore.azure.net"
  resource_group_name = var.resource_group_name
}

resource "azurerm_private_dns_zone_virtual_network_link" "kv" {
  name                  = "link-kv-${var.environment}"
  resource_group_name   = var.resource_group_name
  private_dns_zone_name = azurerm_private_dns_zone.kv.name
  virtual_network_id    = azurerm_virtual_network.spoke.id
  registration_enabled  = false
}
```

PRIVATE DNS ZONE NAMES: QUICK REFERENCE

Service	subresource_names	Private DNS Zone
Key Vault	`["vault"]`	`privatelink.vaultcore.azure.net`
Storage: Blob	`["blob"]`	`privatelink.blob.core.windows.net`
Storage: File	`["file"]`	`privatelink.file.core.windows.net`
Azure SQL	`["sqlServer"]`	`privatelink.database.windows.net`
Azure Container Registry	`["registry"]`	`privatelink.azurecr.io`
Azure OpenAI	`["account"]`	`privatelink.openai.azure.com`
AI Search	`["searchService"]`	`privatelink.search.windows.net`
App Service / Functions	`["sites"]`	`privatelink.azurewebsites.net`

3: MANAGED IDENTITY + RBAC ASSIGNMENT

System-Assigned (simplest)

```
resource "azurerm_linux_function_app" "main" {
  # ...
  identity {
    type = "SystemAssigned"
  }
}

resource "azurerm_role_assignment" "fn_kv_reader" {
  scope                = azurerm_key_vault.main.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azurerm_linux_function_app.main.identity[0].principal_id
}
```

User-Assigned (reusable, recommended)

```
resource "azurerm_user_assigned_identity" "app" {
  name                = "id-app-${var.environment}"
  location            = var.location
  resource_group_name = var.resource_group_name
}

resource "azurerm_role_assignment" "app_blob_contrib" {
  scope                = azurerm_storage_account.main.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_user_assigned_identity.app.principal_id
}
# Attach to any resource:
# identity { type = "UserAssigned"
#   identity_ids = [azurerm_user_assigned_identity.app.id] }
```

Role Principle: Always use the least-privileged built-in role. `Key Vault Secrets User` (read-only) beats `Key Vault Contributor` (control plane). Never assign `Owner` to a managed identity.

4: AZURE FIREWALL FORCED TUNNELING ROUTES

Route table: default route + KMS / Azure AD bypass (prevents broken Windows VMs & auth failures)

```
resource "azurerm_route_table" "spoke" {
  name                          = "rt-spoke-${var.environment}"
  location                      = var.location
  resource_group_name           = var.resource_group_name
  bgp_route_propagation_enabled = false
}

# Default route: send everything to the firewall
resource "azurerm_route" "default" {
  name                   = "DefaultRoute"
  resource_group_name    = var.resource_group_name
  route_table_name       = azurerm_route_table.spoke.name
  address_prefix         = "0.0.0.0/0"
  next_hop_type          = "VirtualAppliance"
  next_hop_in_ip_address = azurerm_firewall.main.ip_configuration[0].private_ip_address
}

# KMS bypass (azkms.core.windows.net:1688), without this Windows VMs fail to activate
resource "azurerm_route" "kms" {
  for_each            = toset(["20.118.99.224/32", "40.83.235.53/32"])
  name                = "KMSBypass-${replace(each.key, "/", "_")}"
  resource_group_name = var.resource_group_name
  route_table_name    = azurerm_route_table.spoke.name
  address_prefix      = each.key
  next_hop_type       = "Internet"
}

# AzureAD bypass (service tag): prevents Managed Identity auth failures
resource "azurerm_route" "aad" {
  name                = "AzureADBypass"
  resource_group_name = var.resource_group_name
  route_table_name    = azurerm_route_table.spoke.name
  address_prefix      = "AzureActiveDirectory"
  next_hop_type       = "Internet"
}
```

Why these two IPs? Azure KMS (`azkms.core.windows.net`, TCP 1688) resolves to 20.118.99.224 and 40.83.235.53. A forced-tunnel 0.0.0.0/0 route sends activation through the firewall, which drops it unless allowed, and Windows VMs fall into notification mode. 168.63.129.16 (DNS, health probes, DHCP) is not affected by UDRs and needs no route.

5: DINE POLICY LIFECYCLE BYPASS

Prevent "Resource was modified after deployment" pipeline failures

```
resource "azurerm_private_dns_zone_virtual_network_link" "kv" {
  # ... standard config ...

  lifecycle {
    ignore_changes = [
      tags,           # DINE policies inject tags after creation
      registration_enabled,
    ]
  }
}

# Same pattern for NSGs: DeployIfNotExists policies
# modify NSG rules or add diagnostic settings:
resource "azurerm_network_security_group" "spoke" {
  # ...
  lifecycle {
    ignore_changes = [security_rule, tags]
  }
}
```

When to use: Any resource in a tenant with Azure Policy (DINE or Modify effects). Without this, every `terraform plan` after policy remediation shows a diff, and applies break the policy-managed state.

6: AZAPI SHARED PRIVATE LINK AUTO-APPROVAL (AI / SEARCH)

Automate the Portal click that blocks every AI RAG deployment

```
# 1. Search requests the link (azurerm can do this part)
resource "azurerm_search_shared_private_link_service" "openai" {
  name               = "spl-openai"
  search_service_id  = azurerm_search_service.main.id
  subresource_name   = "openai_account"
  target_resource_id = azurerm_cognitive_account.openai.id
  request_message    = "Requested by Terraform"
}

# 2. Read the pending connection on the OpenAI side
data "azapi_resource_list" "openai_pe" {
  type                   =
"Microsoft.CognitiveServices/accounts/privateEndpointConnections@2023-05-01"
  parent_id              = azurerm_cognitive_account.openai.id
  response_export_values = ["value"]
  depends_on             = [azurerm_search_shared_private_link_service.openai]
}

# 3. Approve it on the target (OpenAI) resource.
#    Match on the request message, not on "Pending": the description is
#    kept on approval, so re-applies stay idempotent, and the auto-approved
#    private endpoint from your own VNet is never picked up by mistake.
resource "azapi_update_resource" "approve_openai_link" {
  type        = "Microsoft.CognitiveServices/accounts/privateEndpointConnections@2023-05-
01"
  resource_id = one([
    for c in data.azapi_resource_list.openai_pe.output.value : c.id
    if c.properties.privateLinkServiceConnectionState.description == "Requested by
Terraform"
  ])
  body = {
    properties = {
      privateLinkServiceConnectionState = {
        status      = "Approved"
        description = "Requested by Terraform"
      }
    }
  }
}

# Required provider block (azapi 2.x: output is an object, body is HCL):
# terraform { required_providers {
#   azapi = { source = "Azure/azapi", version = "~> 2.0" } }}
```

Why AzAPI and not azurerm? `azurerm_search_shared_private_link_service` only creates the request from the Search side. The approval has to happen on the target OpenAI account, and azurerm has no resource for approving an inbound Cognitive Services connection.

7: KEY VAULT HARDENED CONFIGURATION

Zero public access, soft-delete, purge protection: ISO 27001 / NIS2 baseline

```
resource "azurerm_key_vault" "main" {
  name                          = "kv-${var.project}-${var.environment}"
  location                      = var.location
  resource_group_name           = var.resource_group_name
  tenant_id                     = data.azurerm_client_config.current.tenant_id
  sku_name                      = "standard"

  enable_rbac_authorization     = true   # disable legacy access policies
  soft_delete_retention_days    = 90
  purge_protection_enabled      = true   # mandatory for KRITIS / NIS2
  public_network_access_enabled = false  # Private Endpoint only

  network_acls {
    default_action = "Deny"
    bypass         = "AzureServices"   # allow ARM to reach it for policy
    ip_rules       = []
  }
}
```

8: NIS2 ART. 21 / ISO 27001 CONTROL → TERRAFORM RESOURCE MAP

Control Requirement	NIS2 Art. 21	ISO 27001	Terraform Resource(s)
Network segmentation	§21(2)(i)	A.8.22	`azurerm_subnet` + `azurerm_network_security_group`
Default-deny access control	§21(2)(i)	A.8.20	`azurerm_network_security_rule` priority 4096 Deny
Eliminate public exposure	§21(2)(i)	A.8.21	`azurerm_private_endpoint` + `public_network_access_enabled = false`
Least-privilege identity	§21(2)(i)	A.5.15	`azurerm_user_assigned_identity` + `azurerm_role_assignment`
Encrypt data in transit	§21(2)(h)	A.8.24	`min_tls_version = "TLS1_2"` on all storage/app resources
Secret management / key rotation	§21(2)(h)	A.8.24	`azurerm_key_vault` with `purge_protection_enabled = true`
Audit trail / immutable logging	§21(2)(b)	A.8.15	`azurerm_monitor_diagnostic_setting` → Log Analytics / Storage
Egress traffic filtering	§21(2)(i)	A.8.23	`azurerm_firewall_policy_rule_collection_group` (FQDN allow-list)